

Models of Language and Conversation

Lecture 23: Implementing GPT¹ from Scratch

Building a Language Model in PyTorch

Models of Language and Conversation

Week 7

Week 7

Models of Language and Conversation

Today's Journey

Week 7

What We'll Build Today

Goal

Implement a simplified GPT model from scratch in PyTorch!

Components we'll cover:

1. Tokenization (Byte-Pair Encoding)
2. Embeddings (token + position)
3. Masked Multi-Head Attention
4. Transformer Decoder Block
5. Language Model Head
6. Training Loop

Required Libraries

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from torch.utils.data import Dataset, DataLoader
5 import numpy as np
6 import tiktoken # OpenAI's BPE tokenizer
7
8 # Check for GPU
9 device = 'cuda' if torch.cuda.is_available() else 'cpu'
10 print(f"Using device: {device}")
11
12 # Hyperparameters
13 config = {
14     'vocab_size': 50257, # GPT-2 vocabulary size
15     'd_model': 384, # Embedding dimension
16     'n_layers': 6, # Number of transformer blocks
17     'n_heads': 6, # Number of attention heads
18     'max_seq_len': 256, # Maximum sequence length
19     'dropout': 0.1,
```

Models of Language and Conversation

Required Libraries

```
20 | 'batch_size': 32,  
21 | 'learning_rate': 3e-4,  
22 | 'num_epochs': 10  
23 | }
```

Install: pip install torch tiktoken

Week 7

What is Tokenization?

Tokenization

Converting text into a sequence of integer IDs that the model can process.

Common strategies:

1. **Word-level:** Split on spaces

- Huge vocabulary, can't handle unknown words

2. **Character-level:** Individual characters

Byte-Pair Encoding (BPE)

How BPE works:

1. Start with characters as base vocabulary
2. Find most frequent pair of adjacent tokens
3. Merge this pair into a new token
4. Repeat until desired vocabulary size

Example

Text: "low low low lower lower newest newest"

Iterations:

1. Merge "l" + "o" → "lo"
2. Merge "lo" + "w" → "low"

BPE Tokenization in Code

```
1 import tiktoken
2
3 # Load GPT-2 tokenizer (uses BPE)
4 tokenizer = tiktoken.get_encoding("gpt2")
5
6 # Example text
7 text = "Hello, how are you doing today?"
8
9 # Encode: text → token IDs
10 tokens = tokenizer.encode(text)
11 print("Tokens:", tokens)
12 # Output: [15496, 11, 703, 389, 345, 1804, 1909, 30]
13
14 # Decode: token IDs → text
15 decoded = tokenizer.decode(tokens)
16 print("Decoded:", decoded)
17 # Output: "Hello, how are you doing today?"
18
```

Models of Language and Conversation

BPE Tokenization in Code

```
19 # See individual token strings
20 for token_id in tokens:
21     token_str = tokenizer.decode([token_id])
22     print(f"{token_id}: '{token_str}'")
23
24 # Vocabulary size
25 print(f"Vocab size: {tokenizer.n_vocab}") # 50257
```

Week 7

Creating a Text Dataset

```
1 class TextDataset(Dataset):
2     def __init__(self, text_file, tokenizer, max_seq_len):
3         # Load text
4         with open(text_file, 'r', encoding='utf-8') as f:
5             text = f.read()
6
7         # Tokenize entire text
8         self.tokens = tokenizer.encode(text)
9         self.max_seq_len = max_seq_len
10
11     def __len__(self):
12         # Number of sequences we can extract
13         return len(self.tokens) - self.max_seq_len
14
15     def __getitem__(self, idx):
16         # Get sequence of length max_seq_len + 1
```

Week 7

10

continued...

Creating a Text Dataset

```
17 # (we need +1 for the target)
18 chunk = self.tokens[idx:idx + self.max_seq_len + 1]
19
20 # Input: all but last token
21 x = torch.tensor(chunk[:-1], dtype=torch.long)
22 # Target: all but first token
23 y = torch.tensor(chunk[1:], dtype=torch.long)
24
25 return x, y
26
27 # Usage
28 dataset = TextDataset('shakespeare.txt', tokenizer, config['max_seq_len'])
29 dataloader = DataLoader(dataset, batch_size=config['batch_size'], shuffle=True)
```

Token and Position Embeddings

```
1 class Embeddings(nn.Module):
2     def __init__(self, vocab_size, d_model, max_seq_len, dropout):
3         super().__init__()
4         # Token embeddings: map token IDs to vectors
5         self.token_embed = nn.Embedding(vocab_size, d_model)
6
7         # Position embeddings: encode position information
8         self.pos_embed = nn.Embedding(max_seq_len, d_model)
9
10        self.dropout = nn.Dropout(dropout)
11        self.d_model = d_model
12
13        def forward(self, x):
14            # x shape: (batch_size, seq_len)
15            seq_len = x.size(1)
16
```

Week 7

12

continued...

Token and Position Embeddings

```
17 # Token embeddings
18 tok_emb = self.token_embed(x) # (batch, seq_len, d_model)
19
20 # Position embeddings
21 positions = torch.arange(0, seq_len, device=x.device)
22 pos_emb = self.pos_embed(positions) # (seq_len, d_model)
23
24 # Combine (broadcasting handles batch dimension)
25 embeddings = tok_emb + pos_emb
26
27 return self.dropout(embeddings)
```

Masked Multi-Head Attention

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads, dropout):
3         super().__init__()
4         assert d_model % n_heads == 0
5
6         self.d_model = d_model
7         self.n_heads = n_heads
8         self.head_dim = d_model // n_heads
9
10        # Linear layers for Q, K, V
11        self.q_linear = nn.Linear(d_model, d_model)
12        self.k_linear = nn.Linear(d_model, d_model)
13        self.v_linear = nn.Linear(d_model, d_model)
14
```

Models of Language and Conversation

Masked Multi-Head Attention

```
15 # Output projection
16 self.out_linear = nn.Linear(d_model, d_model)
17 self.dropout = nn.Dropout(dropout)
18
19 def forward(self, x, mask=None):
20     batch_size, seq_len, d_model = x.shape
21
22     # Linear projections and split into heads
23     Q = self.q_linear(x).view(batch_size, seq_len, self.n_heads, self.head_dim)
24     K = self.k_linear(x).view(batch_size, seq_len, self.n_heads, self.head_dim)
25     V = self.v_linear(x).view(batch_size, seq_len, self.n_heads, self.head_dim)
26
27     # Transpose for attention: (batch, n_heads, seq_len, head_dim)
28     Q = Q.transpose(1, 2)
```

Week 7

15

...continued...

Masked Multi-Head Attention

```
29 | K = K.transpose(1, 2)  
30 | V = V.transpose(1, 2)
```

Week 7

Attention Computation (cont.)

```
1 # Scaled dot-product attention
2 # Scores: (batch, n_heads, seq_len, seq_len)
3 scores = torch.matmul(Q, K.transpose(-2, -1)) / np.sqrt(self.head_dim)
4
5 # Apply causal mask (prevent attending to future tokens)
6 if mask is not None:
7     scores = scores.masked_fill(mask == 0, float('-inf'))
8
9 # Softmax to get attention weights
10 attn_weights = F.softmax(scores, dim=-1)
11 attn_weights = self.dropout(attn_weights)
12
13 # Apply attention to values
14 # Output: (batch, n_heads, seq_len, head_dim)
15 attn_output = torch.matmul(attn_weights, V)
16
17 # Concatenate heads
```

17

Week 7

continued...

Attention Computation (cont.)

```
18 | attn_output = attn_output.transpose(1, 2).contiguous()
19 | attn_output = attn_output.view(batch_size, seq_len, d_model)
20 |
21 | # Final linear projection
22 | output = self.out_linear(attn_output)
23 |
24 | return output
```

Creating the Causal Mask

```
1 def create_causal_mask(seq_len, device):
2     """
3     Create a causal (lower-triangular) mask for autoregressive generation.
4
5     Returns:
6     mask: (seq_len, seq_len) with 1s on and below diagonal, 0s above
7     """
8     mask = torch.tril(torch.ones(seq_len, seq_len, device=device))
```

Key insight: Each position can only attend to itself
and previous positions!

Feed-Forward Network

```
1 | class FeedForward(nn.Module):
```

Why GELU (Gaussian Error Linear Unit)?

- Smooth, non-monotonic activation
- Used in BERT and GPT
- Slight improvement over ReLU for language models

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot 2 [1 + \text{erf}(-\frac{x}{\sqrt{2}})]$$

Week 7

Models of Language and Conversation

Transformer Decoder Block

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, d_model, n_heads, dropout):
3         super().__init__()
4         self.attention = MultiHeadAttention(d_model, n_heads, dropout)
5         self.feed_forward = FeedForward(d_model, dropout)
6
7         # Layer normalization (applied before sub-layers in GPT)
8         self.ln1 = nn.LayerNorm(d_model)
```

Note: GPT uses *pre-norm* (LayerNorm before sub-layers), while original Transformer used *post-norm*.

Complete GPT Model

```
1 class GPT(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, n_heads, max_seq_len, dropout):
3         super().__init__()
4         self.max_seq_len = max_seq_len
5
6         # Embeddings
7         self.embeddings = Embeddings(vocab_size, d_model, max_seq_len, dropout)
8
9         # Transformer blocks
10        self.blocks = nn.ModuleList([
11            TransformerBlock(d_model, n_heads, dropout)
12        for _ in range(n_layers)
13        ])
14
15        # Final layer norm
```

Complete GPT Model

```
16 self.ln_f = nn.LayerNorm(d_model)
17
18 # Language model head (projects to vocabulary)
19 self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
20
21 # Initialize weights
22 self.apply(self._init_weights)
23
24 def _init_weights(self, module):
25     if isinstance(module, nn.Linear):
26         torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
27         if module.bias is not None:
28             torch.nn.init.zeros_(module.bias)
29     elif isinstance(module, nn.Embedding):
30         torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

GPT Forward Pass

```
1 def forward(self, x, targets=None):
2     # x shape: (batch_size, seq_len)
3     seq_len = x.size(1)
4
5     # Create causal mask
6     mask = create_causal_mask(seq_len, x.device)
7
8     # Embeddings
9     x = self.embeddings(x) # (batch, seq_len, d_model)
10
11    # Apply transformer blocks
12    for block in self.blocks:
13        x = block(x, mask)
14
15    # Final layer norm
16    x = self.ln_f(x)
17
18    # Project to vocabulary
19    logits = self.lm_head(x) # (batch, seq_len, vocab_size)
```

24

Week 7

continued...

Models of Language and Conversation

GPT Forward Pass

```
20
21     # Compute loss if targets provided
22     loss = None
23     if targets is not None:
24         # Flatten for cross-entropy
25         loss = F.cross_entropy(
26             logits.view(-1, logits.size(-1)),
27             targets.view(-1)
28         )
29
30     return logits, loss
```

Week 7

25

...continued

Training Loop

```
1 # Initialize model
2 model = GPT(
3     vocab_size=config['vocab_size'],
4     d_model=config['d_model'],
5     n_layers=config['n_layers'],
6     n_heads=config['n_heads'],
7     max_seq_len=config['max_seq_len'],
8     dropout=config['dropout']
9 ).to(device)
10
11 # Optimizer (AdamW is used for GPT)
12 optimizer = torch.optim.AdamW(
13     model.parameters(),
14     lr=config['learning_rate'],
15     betas=(0.9, 0.95),
```

Training Loop

```
16 weight_decay=0.1
17 )
18
19 # Training loop
20 model.train()
21 for epoch in range(config['num_epochs']):
22     total_loss = 0
23     for batch_idx, (x, y) in enumerate(dataloader):
24         x, y = x.to(device), y.to(device)
25
26         # Forward pass
27         logits, loss = model(x, targets=y)
28
29         # Backward pass
30         optimizer.zero_grad()
```

Models of Language and Conversation

Training Loop

```
31 | loss.backward()  
32 | torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  
33 | optimizer.step()  
34 |  
35 | total_loss += loss.item()  
36 |  
37 | avg_loss = total_loss / len(dataloader)  
38 | print(f"Epoch {epoch+1}/{config['num_epochs']}, Loss: {avg_loss:.4f}")
```

Week 7

28

...continued

- Warmup for first few thousand steps
- Cosine decay afterwards

3. **AdamW optimizer**

- Adam with decoupled weight decay
- Better than standard Adam for transformers

4. **Batch size**

Greedy Decoding

```
1  @torch.no_grad()
2  def generate_greedy(model, tokenizer, prompt, max_new_tokens=50):
3      model.eval()
4
5      # Encode prompt
6      tokens = tokenizer.encode(prompt)
7      x = torch.tensor([tokens], dtype=torch.long, device=device)
8
9      for _ in range(max_new_tokens):
10         # Get predictions (crop to max_seq_len if needed)
11         x_crop = x[:, -model.max_seq_len:]
12         logits, _ = model(x_crop)
13
14         # Focus on last token's predictions
15         logits = logits[:, -1, :] # (batch, vocab_size)
```

Greedy Decoding

```
16
17 # Get token with highest probability
18 next_token = torch.argmax(logits, dim=-1, keepdim=True)
19
20 # Append to sequence
21 x = torch.cat([x, next_token], dim=1)
22
23 # Stop if we generate end-of-sequence token
24 if next_token.item() == tokenizer.eot_token:
25     break
26
27 # Decode and return
28 generated_text = tokenizer.decode(x[0].tolist())
29 return generated_text
30
```

Models of Language and Conversation

Greedy Decoding

```
31 # Example usage
32 prompt = "Once upon a time"
33 generated = generate_greedy(model, tokenizer, prompt, max_new_tokens=100)
34 print(generated)
```

Week 7

32

...continued

2. Temperature Sampling

- Scale logits by temperature T
- $T < 1$: More conservative (peaked distribution)
- $T > 1$: More random (flat distribution)

3. Top-k Sampling

- Sample from top k most likely tokens
- Typical: $k = 40$

Top-k and Nucleus Sampling

```
1 def sample_next_token(logits, temperature=1.0, top_k=None, top_p=None):
2     """
3     Sample next token from logits with various strategies.
4
5     Args:
6     logits: (vocab_size,) unnormalized log probabilities
7     temperature: Temperature for sampling
8     top_k: If set, only sample from top k tokens
9     top_p: If set, only sample from nucleus (top-p)
10    """
11    # Apply temperature
12    logits = logits / temperature
13
14    # Top-k filtering
15    if top_k is not None:
16        top_k = min(top_k, logits.size(-1))
```

Top-k and Nucleus Sampling

```
17 indices_to_remove = logits < torch.topk(logits, top_k)[0][..., -1, None]
18 logits[indices_to_remove] = float('-inf')
19
20 # Nucleus (top-p) filtering
21 if top_p is not None:
22     sorted_logits, sorted_indices = torch.sort(logits, descending=True)
23     cumulative_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
24
25     # Remove tokens with cumulative probability above threshold
26     sorted_indices_to_remove = cumulative_probs > top_p
27     sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
28     sorted_indices_to_remove[..., 0] = 0
29
30 indices_to_remove = sorted_indices[sorted_indices_to_remove]
31 logits[indices_to_remove] = float('-inf')
32
```

Top-k and Nucleus Sampling

```
33 # Sample from distribution
34 probs = F.softmax(logits, dim=-1)
35 next_token = torch.multinomial(probs, num_samples=1)
36
37 return next_token
```

Week 7

Complete Generation Function

```
1 @torch.no_grad()
2 def generate(model, tokenizer, prompt, max_new_tokens=100,
3     temperature=1.0, top_k=40, top_p=0.9):
4     model.eval()
5
6     tokens = tokenizer.encode(prompt)
7     x = torch.tensor([tokens], dtype=torch.long, device=device)
8
9     for _ in range(max_new_tokens):
10        x_crop = x[:, -model.max_seq_len:]
11        logits, _ = model(x_crop)
12        logits = logits[:, -1, :] # Last token
13
14        # Sample next token
15        next_token = sample_next_token(
16            logits[0],
17            temperature=temperature,
```

Complete Generation Function

```
18     top_k=top_k,  
19     top_p=top_p  
20 )  
21  
22     x = torch.cat([x, next_token.unsqueeze(0)], dim=1)  
23  
24     if next_token.item() == tokenizer.eot_token:  
25         break  
26  
27     return tokenizer.decode(x[0].tolist())  
28  
29     # Creative generation  
30     text = generate(model, tokenizer, "The AI revolution",  
31         temperature=0.8, top_p=0.9)  
32     print(text)
```

Model Size vs. Performance

1 | 10M → 100M → 1B → 10B → 100B+

Practical model sizes for different use cases:

- **10M-100M:** Learning/experimentation, simple tasks
- **100M-1B:** Specialized domains, resource-constrained
- **1B-10B:** General-purpose, good quality
- **10B-100B+:** State-of-the-art performance

Models of Language and Conversation

Computational Requirements

Training a 125M parameter GPT:

Training Time	1-2 days (single GPU)
Training Data	$\sim$ 10-100 GB text
Total Compute	$\sim$ 100 GPU-hours

Scaling up to GPT-3 (175B):

- 1000x more parameters
- $\sim$ 10,000x more compute needed

- Reduce batch size
- Reduce sequence length
- Use gradient accumulation
- Enable mixed precision

3. Poor generation quality

- Train longer
- Use larger model

2. Training techniques

- Learning rate warmup and decay
- Gradient accumulation
- Mixed precision training (FP16/BF16)

3. Data improvements

- Data deduplication
- Quality filtering

- github.com/karpathy/nanoGPT

\item **Andrej Karpathy's "Let's build GPT" video**

- Excellent step-by-step tutorial
- [YouTube](#)

\item **HuggingFace Transformers**

- Production-ready implementations
- huggingface.co/transformers

Hands-On Exercise

Your Task

Implement and train a small GPT model on your own text corpus!

Steps:

1. Choose a dataset (Shakespeare, Wikipedia, your own text)
2. Set up the data pipeline
3. Initialize the model (start small: 6 layers, 384 d_model)
4. Train for 10-20 epochs
5. Experiment with generation
6. Try different sampling strategies

- Token + position embeddings
- Masked multi-head attention
- Feed-forward networks

3. Training requires care

- Good data, proper hyperparameters
- Gradient clipping, learning rate schedules

Readings

Required Readings

1. **Vaswani et al. (2017)** - "Attention is All You Need" \ [\[ArXiv\]](#)
2. **Radford et al. (2018)** - "Improving Language Understanding by Generative Pre-Training" \ [\[PDF\]](#)

Code Resources

- **nanoGPT** by Andrej Karpathy \ github.com/karpathy/nanoGPT
- **The Annotated Transformer** \

Week 8: No Classes - Instructor Away

Use this week to:

- Complete the GPT implementation exercise
- Catch up on readings
- Work on assignments
- Experiment with different architectures

Week 9: RAG & Mixture of Experts

- Retrieval Augmented Generation
- Mixture of Experts architectures

Models of Language and Conversation

Questions?
Happy Coding!

Week 7